

RELATÓRIO CONSOLIDADO DE REVISÃO TÉCNICA — PROJETO ESCALAS

Revisão de Arquitetura, Frontend e Testes — Ondas 1 e 2 de Melhorias

09 de setembro de 2026

1. Sumário Executivo

O presente relatório consolida a revisão técnica completa realizada sobre a base de código do sistema Escalas, desenvolvido para o Curso Técnico em Informática do SENAC-RS (Unidade Gravataí). O projeto foi concebido para substituir o controle operacional de escalas de docentes baseado em planilhas eletrônicas descentralizadas por uma aplicação web transacional com integridade referencial, regras de negócio automatizadas e histórico auditável.

A avaliação estrutural identificou que o sistema possui fundações arquiteturais sólidas e substancialmente acima da média observada em aplicações acadêmicas equivalentes: backend estruturado em camadas claras, erros de domínio tipados e mapeados centralmente, decisões de projeto documentadas formalmente em registros de decisão de arquitetura (ADRs) e uma suíte de testes de integração robusta cobrindo os principais critérios de aceitação de negócio.

Não obstante essa maturidade inicial, a auditoria revelou três deficiências estruturais de alta severidade: a ausência de migrações gerenciadas de banco de dados, que submetia o sistema ao risco de corrupção ou perda de dados em alterações de esquema; a existência de páginas monolíticas no frontend com alta densidade de responsabilidades acumuladas, destacando-se o componente de escala com 75 KB; e um defeito crítico de serialização temporal no módulo utilitário de formatação, capaz de provocar deslocamento retroativo de datas em clientes operando no fuso horário do Brasil.

Para mitigar de forma sistemática tais vulnerabilidades, foram estruturadas e concluídas as Ondas 1 e 2 de Melhorias, resultando na entrega de vinte artefatos de código, configuração e testes integrados ao ambiente de versionamento. O risco residual concentra-se agora exclusivamente na necessidade de decomposição dos monólitos do frontend e no desacoplamento de parâmetros de negócio hoje fixos em código-fonte, intervenções mapeadas nas etapas subsequentes deste plano.

2. Escopo e Método

A revisão técnica abrangeu três eixos complementares de engenharia de software: a arquitetura do backend, a organização do frontend e a eficácia da suíte de testes automatizados. O escopo compreendeu a leitura analítica e a inspeção estrutural de todos os artefatos versionados no repositório.

O procedimento metodológico baseou-se na leitura direta do código-fonte, arquivos de manifesto, configurações de compilação, especificações funcionais e suítes de teste. Todas as conclusões e diagnósticos apresentados neste documento decorrem da verificação estrita do comportamento do código existente, sem inferências hipotéticas ou suposições externas.

Os módulos do backend, rotas da API, esquemas Pydantic, serviços de domínio, modelos ORM, configurações e suítes de teste do backend foram lidos e analisados integralmente. No ambiente do frontend, os componentes compartilhados, clientes HTTP, utilitários, definições de tipos e arquivos de teste foram inspecionados em detalhe, ao passo que os dois arquivos de maior dimensão — *EscalaPage.tsx* (75 KB) e *CadastrrosPage.tsx* (35 KB) — foram avaliados quanto à sua topologia interna, padrões de gerenciamento de estado, ciclo de vida e concentração de acoplamento funcional.

3. Visão Geral do Repositório

A árvore de diretórios do projeto organiza-se em três macroestruturas funcionais: a aplicação backend (*app/*), a aplicação cliente (*frontend/*) e a suíte de testes automatizados (*tests/*), além das especificações funcionais no padrão OpenSpec (*openspec/*) e das decisões arquiteturais na raiz.

Caminho / Artefato	Descrição Funcional	Tamanho Aproximado
app/api/routes/	Controladores HTTP (professores, turmas, ucs, alocaoes, health)	12 KB
app/services/alocacoes.py	Motor de regras de negócio, alocações e validações de conflito	25 KB
app/models/ e app/schemas/	Modelos SQLAlchemy e contratos de entrada/saída Pydantic	10 KB

Caminho / Artefato	Descrição Funcional	Tamanho Aproximado
frontend/src/pages/EscalaPage.tsx	Visão operacional da escala, matrizes de calendário e triagem	75 KB
frontend/src/pages/CadastrosPage.tsx	Interfaces de formulários para entidades e recorrências	35 KB
frontend/src/components/EscalaActionDrawer.tsx	Gaveta contextual de ações com validações preventivas	14 KB
frontend/src/lib/escalaSignals.ts	Funções puras de cálculo de semáforo operacional	4 KB
tests/test_escalas_api.py	Suíte de testes de integração de regras de negócio (CA-01 a CA-09)	23 KB
ADR01.txt, ADR02.txt, ADR03.txt	Registros de decisão arquitetural do sistema	6 KB

Durante a varredura da raiz do projeto, identificaram-se itens de higiene operacional que demandam saneamento: a presença do arquivo *requirements-backup.txt* vazio, a permanência de dois arquivos binários em formato PDF na raiz do controle de versão e a coexistência simultânea de três configurações de compilação do Vite (*vite.config.ts*, *vite.config.js* e *vite.config.d.ts*), originadas de processos espúrios de transpilação.

4. Revisão de Arquitetura (Backend)

O backend adota o padrão em camadas direcionadas: os controladores HTTP (*app/api/routes*) delegam a orquestração exclusivamente para a camada de serviços (*app/services*), que manipula os modelos de persistência (*app/models*) e assegura o cumprimento dos contratos estruturados via Pydantic (*app/schemas*). A infraestrutura de

conexão reside em *app/db/session* e as variáveis de ambiente são centralizadas em *app/core/config*.

Dentre os pontos fortes, destaca-se a modelagem rigorosa de exceções de domínio através da classe base *RegraDeNegocioError* e suas especializações: *ValidacaoDeNegocioError* (mapeada para HTTP 400), *ConflitoDeNegocioError* (HTTP 409) e *EntidadeNaoEncontradaError* (HTTP 404), tratadas de forma centralizada pelo middleware de exceções. A configuração segue as diretrizes do manifesto 12-Factor via *pydantic-settings*, assegurando o isolamento de parâmetros operacionais. A integridade dos dados é garantida a nível relacional por meio de restrições de unicidade, *CheckConstraint* impedindo que o professor titular seja simultaneamente cadastrado como substituto, e índice único parcial garantindo a unicidade por turma, data e turno apenas quando a alocação não for forçada (*forcada = 0*). Essa abordagem materializa as diretrizes documentadas nos ADRs 01, 02 e 03.

Não obstante a solidez estrutural, foram identificadas oportunidades de melhoria técnica:

- Ausência de migrações versionadas: o uso exclusivo de *Base.metadata.create_all()* no evento de inicialização inviabiliza a evolução controlada do banco de dados em produção e impõe risco de perda de dados.
- CRUD incompleto: ausência de manipuladores de atualização parcial (*PATCH/PUT*) para professores, turmas e unidades curriculares, obrigando exclusões seguidas de recriações para corrigir dados simples.
- Acoplamento na validação de turno: a função de validação de turnos foi alocada em *services/alocacoes.py* e importada por *services/turmas.py*, gerando acoplamento indevido entre domínios.
- Dialeto do índice parcial restrito ao SQLite: o uso de cláusula expressa vinculada ao dialeto SQLite exigirá readequação quando da migração para outros bancos relacionais.
- Validação declarativa frouxa nos esquemas: os contratos Pydantic originais não impunham restrições de comprimento mínimo nem intervalos numéricos diretamente nas definições de campos.
- Endpoint de verificação operacional simplificado: a rota de verificação de integridade (*health check*) responde de forma estática sem testar a conectividade ativa com o mecanismo de banco de dados.
- Ausência de cabeçalhos CORS: o backend depende integralmente do proxy do ambiente de desenvolvimento do Vite, quebrando se consumido de origens cruzadas.
- Higiene de código: presença de instrução órfã (*db.get_bind()*) no serviço de turmas, mensagens de erro sem caracteres acentuados e assimetria de nomenclatura entre rotas em inglês e serviços em português.

5. Revisão do Frontend

O frontend foi concebido sobre uma base moderna composta por React 19, Vite, TypeScript, React Router 7 e Vitest. A camada de comunicação HTTP (*lib/api.ts*) é tipada e isola o tratamento de respostas sem corpo (HTTP 204) e erros estruturados (*ApiError*). O motor de cálculo de semáforos operacionais (*lib/escalaSignals.ts*) opera de forma exemplar como um módulo isolado de funções puras desacoplado da interface visual. Os componentes compartilhados apresentam implementação adequada de atributos de acessibilidade (ARIA), enquanto o componente *EscalaActionDrawer.tsx* destaca-se pela execução de validações preventivas no cliente alinhadas às regras do servidor.

Entretanto, o frontend apresenta pontos críticos de acúmulo de complexidade:

- Monólito em *EscalaPage.tsx* (75 KB): o componente absorve simultaneamente a persistência em armazenamento local e de sessão, chamadas de rede paralelas, filtragens operacionais combinatórias, renderização de múltiplos modos de calendário, lógica de seleção em lote e consultas customizadas por período.
- Monólito em *CadastrrosPage.tsx* (35 KB): congrega cinco formulários operacionais distintos e executa a recarga integral de todas as coleções de entidades após qualquer operação atômica de criação ou exclusão.
- Duplicação de rotinas auxiliares: funções utilitárias como leitura de mensagens de erro e instanciação de estruturas de alocação encontram-se repetidas em múltiplos arquivos.
- Duplicação de regras de negócio: a validação preventiva no cliente replica a lógica de validação do backend sem uma fonte única compartilhada de contratos.
- Ausência de camada gerenciada de estado de servidor: as requisições utilizam encadeamento manual de efeitos colaterais sem política de cache ou sincronização automática.
- Agregação de dados no cliente no painel analítico: a página de indicadores requisita a totalidade das alocações dos três turnos para calcular cargas horárias no navegador, utilizando elementos sem semântica estruturada de dados.
- Inconsistências tipográficas: múltiplos textos exibidos na interface apresentam supressão de caracteres acentuados.

5.1. O Defeito Crítico de Fuso Horário

O problema mais grave identificado no frontend residia nas rotinas de geração de intervalos de datas no utilitário *format.ts* (especificamente em *buildDateRangeFromBounds* e *buildFallbackDates*). As funções utilizavam a conversão nativa *toISOString().slice(0, 10)* sobre instâncias de data construídas no contexto local.

No fuso horário oficial do Brasil (UTC-3), qualquer instância correspondente à meia-noite local (00:00) de um determinado dia sofre um recuo temporal de três horas ao ser convertida para o horário universal (UTC), resultando nas 21:00 do dia imediatamente anterior. Em termos práticos, ao processar a data 01/09/2026 00:00:00 (UTC-3), a serialização produzia 2026-08-31T21:00:00Z, e o truncamento de texto extraía a string

errônea "2026-08-31". Esse comportamento causava a exibição de dias incorretos nas matrizes de escala e falhas severas na virada de meses.

6. Revisão dos Testes

A suíte de testes do backend estruturada em *tests/test_escalas_api.py* demonstra cobertura exemplar das regras de negócio, organizando-se por meio de casos de aceitação numerados sequencialmente (CA-01 a CA-09). Os testes validam rigorosamente cenários de duplicidade de alocação, sobreposições forçadas justificadas, rejeição de justificativas insuficientes, conflitos de horário de docentes, impedimento de titularidade concorrente com substituição, validação de turnos, ordenação de registros, cálculo explícito de lacunas operacionais, bloqueio de dias não letivos, operações em lote com pré-visualização e confirmação, alocações recorrentes com bloqueios parciais, teto de carga horária e sinalização de semáforos visuais.

A infraestrutura de teste em *tests/conftest.py* adota padrões consolidados de isolamento: uso de banco SQLite em memória, pool estático de conexões (*StaticPool*), imposição mandatória de chaves estrangeiras via *PRAGMA foreign_keys=ON* e limpeza completa do esquema após cada execução.

As lacunas identificadas nos testes foram as seguintes:

- Concentração excessiva: arquivo único de testes de integração contendo 23 KB de código acoplado.
- Ausência de testes unitários no backend: inexistência de validação isolada das funções de serviço fora do contexto de requisição HTTP via *TestClient*.
- Ausência de métricas de cobertura: falta de integração com ferramentas de análise de cobertura de código (*pytest-cov*).
- Vácuo de cobertura no frontend: ausência absoluta de testes unitários para o módulo *format.ts* (onde residia o defeito de fuso horário), para o motor de semáforos *escalaSignals.ts* e para o cliente *api.ts*.

7. Problemas Priorizados

Prioridade	Problema Identificado	Impacto Técnico e Operacional
Alta	Defeito de conversão de fuso horário em <i>format.ts</i>	Exibição de datas incorretas e corrupção de escala no calendário
Alta	Ausência de migrações estruturadas de banco de dados	Risco crítico de perda de dados em atualizações de esquema
Alta	Componente monolítico <i>EscalaPage.tsx</i> (75 KB)	Fragilidade de manutenção e alto risco de regressões funcionais

Prioridade	Problema Identificado	Impacto Técnico e Operacional
Média	Ausência de esteira de Integração Contínua (CI)	Possibilidade de integração de código defeituoso ao branch principal
Média	Falta de automação de padronização (ESLint / Prettier)	Degradação da consistência estrutural do código cliente
Média	Inexistência de medição formal de cobertura de testes	Falta de visibilidade sobre áreas não testadas do código
Média	Tipagem fraca do campo de turno (string genérica)	Possibilidade de passagem de valores inválidos em tempo de execução
Baixa	Acúmulo de arquivos órfãos e binários na raiz	Poluição do repositório e aumento desnecessário do clone
Baixa	Supressão de acentos gráficos na interface e erros	Prejuízo à experiência de uso e ao acabamento profissional
Baixa	Verificação de saúde operacional estática	Incapacidade de detectar indisponibilidade da camada de persistência

8. Onda 1 — Correções Entregues

A Onda 1 teve como foco a eliminação de defeitos funcionais críticos, a aplicação de tipagem estrita de tipos primitivos e a criação de testes unitários no frontend.

O defeito de fuso horário foi solucionado em *format.ts* por meio da criação da função *toIsoDate()*, que extrai diretamente os componentes locais de ano, mês e dia (*getFullYear*, *getMonth*, *getDate*) sem intermediar a serialização em UTC. Adicionou-se também a rotina *parseIsoDate()*, eliminando a inconsistência do construtor padrão do JavaScript ao interpretar datas sem hora explícita.

A integridade da tipagem de turnos foi estabelecida no frontend aplicando-se a união literal *Turno* aos contratos de interface que antes utilizavam tipos genéricos. No backend, os esquemas de entrada de dados (*AlocacaoCreate*, *AlocacaoBulkCreateRequest* e *TurmaCreate*) passaram a exigir o enum nativo *Turno*, garantindo que requisições com valores inválidos sejam rejeitadas na borda da API com código HTTP 422 e documentadas adequadamente no Swagger OpenAPI. Foram elaborados testes unitários completos cobrindo os cenários de virada de mês e o cálculo exato dos semáforos visuais.

Arquivo Entregue	Destino no Repositório	Finalidade da Modificação
frontend-src-lib-format.ts	frontend/src/lib/format.ts	Correção definitiva do cálculo local de datas e fusos
frontend-src-types-api.ts	frontend/src/types/api.ts	Tipagem estrita de <i>Turno</i> em contratos da API
frontend-src-lib-format.test.ts	frontend/src/lib/format.test.ts	Suíte de testes de regressão temporal e intervalos
frontend-src-lib-escalaSignals.test.ts	frontend/src/lib/escalaSignals.test.ts	Testes unitários do motor de sinais e ações operacionais
app-schemas-alocacao.py	app/schemas/alocacao.py	Validação de enum de turno na entrada de alocações
app-schemas-turma.py	app/schemas/turma.py	Validação de enum de turno no cadastro de turmas
frontend-vite.config.ts	frontend/vite.config.ts	Consolidação de compilação e eliminação de duplicidades
GUIA-APLICACAO-ONDA-1.md	—	Instruções detalhadas de aplicação dos arquivos da Onda 1

9. Onda 2 — Infraestrutura Entregue

A Onda 2 estabeleceu a infraestrutura de controle de versão de banco de dados, governança de qualidade de código e automação de integração contínua.

Implementou-se o framework de migrações Alembic, configurando o ambiente de execução para importar dinamicamente a string de conexão a partir de *app.core.config*, assegurando uma única fonte da verdade para credenciais. A migração inicial (*0001_inicial.py*) materializa as quatro tabelas do sistema com suas respectivas restrições relacionais, chaves estrangeiras e índices parciais. O ciclo de vida da

aplicação (*app/main.py*) foi refatorado para remover a criação destrutiva de tabelas na inicialização.

Configurou-se uma esteira de Integração Contínua via GitHub Actions executando dois trabalhos paralelos: validação do backend sob Python 3.12 com imposição de cobertura mínima de 70% pelo *pytest-cov*, e validação do frontend sob Node 22 com checagem rigorosa de tipos, compilação de produção e execução da suíte Vitest. Adicionalmente, padronizou-se o ferramental de estilo no cliente com ESLint 9 e Prettier.

Arquivo Entregue	Destino no Repositório	Finalidade da Modificação
alembic.ini	alembic.ini	Arquivo mestre de configuração do Alembic
alembic-env.py	alembic/env.py	Ambiente de execução integrado às configurações da aplicação
alembic-script.py.mako	alembic/script.py.mako	Modelo estrutural para geração de novas migrações
alembic-versions-0001-inicial.py	alembic/versions/0001_inicial.py	Migração inicial com esquemas, chaves e restrições
app-main.py	app/main.py	Remoção de <i>create_all</i> no ciclo de vida da API
pyproject.toml	pyproject.toml	Inclusão de dependências e métricas de cobertura
github-workflows-ci.yml	.github/workflows/ci.yml	Pipeline de automação de testes e checagens (CI)
frontend-eslint.config.js	frontend/eslint.config.js	Configuração flat do linter para TypeScript e React
frontend-prettierrc	frontend/.prettierrc	Definição de regras de formatação de código
frontend-prettierignore	frontend/.prettierignore	Exclusão de diretórios de compilação da formatação
frontend-package.json	frontend/package.json	Scripts de lint, formatação e novas dependências
GUIA-APLICACAO-ONDA-2.md	—	Instruções detalhadas de aplicação dos arquivos da

Arquivo Entregue	Destino no Repositório	Finalidade da Modificação
		Onda 2

10. Ondas 3 e 4 — Planejamento

10.1. Onda 3 — Decomposição Estrutural do Frontend

- Decomposição de *EscalaPage.tsx*: extração de cinco hooks especializados: *useEscalaData* (carregamento de coleções), *useEscalaFilters* (filtros operacionais e presets), *useEscalaPersistence* (gerenciamento de armazenamento local e de sessão), *useBulkRemoval* (orquestração de pré-visualização e exclusão em lote) e *useTurmaPeriodo* (consultas analíticas dedicadas).
- Isolamento de subcomponentes: migração dos componentes internos *WeeklyMatrix* e *ActionStack* para a pasta de componentes independentes.
- Modularização de *CadastrrosPage.tsx*: segregação da interface em formulários dedicados: *ProfessorForm*, *UcForm*, *TurmaForm*, *AlocacaoForm* e *BulkRecurringForm*.
- Invalidação granular de cache: substituição da recarga integral de entidades por mutações seletivas da entidade afetada.
- Saneamento de valores sentinela: eliminação do valor numérico *0* como indicador de ausência de substituto, utilizando *null*.
- Endpoint analítico dedicado: criação da rota *GET /professores/carga* no backend para cálculo consolidado de horas no banco, desonerando o cliente.

10.2. Onda 4 — Evolução e Prontidão Operacional

- Externalização de constantes de negócio: parametrização de valores fixos como carga horária padrão por alocação (3 horas), limite anual de horas (300h) e cálculo de feriados móveis.
- Completude de operações CRUD: implementação de manipuladores *PATCH/PUT* para atualização de cadastros de professores, turmas e disciplinas.
- Compatibilidade com bancos corporativos: adaptação da cláusula do índice parcial para garantir interoperabilidade transparente caso haja migração do SQLite para PostgreSQL.
- Controle de autenticação e autorização: introdução de camadas de segurança e identificação de perfis de acesso previamente à disponibilização do sistema para múltiplos operadores concorrentes.

11. Mapa de Arquivos Entregues

Pasta de Origem no Drive	Nome do Arquivo no Drive	Caminho de Destino no Repositório
escalas-onda-1	frontend-src-lib-format.ts	frontend/src/lib/format.ts
escalas-onda-1	frontend-src-types-api.ts	frontend/src/types/api.ts
escalas-onda-1	frontend-src-lib-format.test.ts	frontend/src/lib/format.test.ts
escalas-onda-1	frontend-src-lib-escalaSignals.test.ts	frontend/src/lib/escalaSignals.test.ts
escalas-onda-1	app-schemas-alocacao.py	app/schemas/alocacao.py
escalas-onda-1	app-schemas-turma.py	app/schemas/turma.py
escalas-onda-1	frontend-vite.config.ts	frontend/vite.config.ts
escalas-onda-1	GUIA-APLICACAO-ONDA-1.md	GUIA-APLICACAO-ONDA-1.md
escalas-onda-2	alembic.ini	alembic.ini
escalas-onda-2	alembic-env.py	alembic/env.py
escalas-onda-2	alembic-script.py.mako	alembic/script.py.mako
escalas-onda-2	alembic-versions-0001-inicial.py	alembic/versions/0001_inicial.py
escalas-onda-2	app-main.py	app/main.py
escalas-onda-2	pyproject.toml	pyproject.toml
escalas-onda-2	github-workflows-ci.yml	.github/workflows/ci.yml
escalas-onda-2	frontend-eslint.config.js	frontend/eslint.config.js
escalas-onda-2	frontend-prettierrc	frontend/.prettierrc
escalas-onda-2	frontend-prettierignore	frontend/.prettierignore
escalas-onda-2	frontend-package.json	frontend/package.json
escalas-onda-2	GUIA-APLICACAO-ONDA-2.md	GUIA-APLICACAO-ONDA-2.md

12. Comandos de Aplicação

Para aplicar as modificações entregues e validar a integridade dos ambientes, execute as sequências de comandos estruturadas a seguir:

Remoção de artefatos espúrios e higiene do repositório:

bash

```
# Na raiz do repositóriorm -f frontend/vite.config.js frontend/vite.config.d.tsrm -f req
```

Atualização e validação do Backend:

bash

```
# Instalação das dependências e ferramentas de desenvolvimento  
pip install -e ".[dev]"  
# Execução da migração inicial do banco de dados  
alembic upgrade head  
# Execução dos testes automatizados com medição de cobertura  
pytest --cov=app --cov-repor
```

Atualização e validação do Frontend:

bash

```
# No diretório frontend/cd frontend  
# Instalação das novas dependências de formatação e análise estática  
npm install  
# Verificação e formatação automática de código  
npm run lint  
npm run format  
# Execução da suíte de testes unitários  
npm run test -- --run  
# Verificação de compilação estrita para produção  
npm run build
```

13. Considerações Finais

O sistema Escalas apresenta maturidade de engenharia considerável, refletida na estruturação limpa de suas camadas de serviço, na formalização de decisões arquiteturais e na cobertura assertiva de suas regras de negócio por meio de testes automatizados. A execução das Ondas 1 e 2 de Melhorias eliminou o risco iminente de corrupção temporal de dados no calendário, instituiu o versionamento formal do esquema relacional via Alembic e garantiu a governança do código através de pipelines de Integração Contínua e ferramentas de padronização estática.

Com a infraestrutura de sustentação plenamente consolidada, o foco técnico prioritário passa a ser a execução da Onda 3, orientada à decomposição dos componentes monolíticos do frontend, assegurando a manutenibilidade e a longevidade operacional do software.

Documento elaborado em 09 de setembro de 2026. As informações contidas são de responsabilidade do solicitante.